

Lecture 5: Debugging & Exceptions

How to Avoid Debugging

First Defense: Make Bugs Impossible

- **Static Checking**
compile (type errors) اكتشاف الأخطاء وقت الـ
 - **Dynamic Checking**
ArrayIndexOutOfBoundsException (زي) اكتشاف الأخطاء وقت التشغيل
 - **Immutability**
الكائن لا يتغير بعد إنشائه → يقلل الأخطاء
مثال: String immutable.
 - **final keyword**
Documentation + يمنع تغيير المتغير أو المرجع → أمان أكثر
-

Second Defense: Localize Bugs

- الهدف: نخلي الخطأ في مكان صغير
 - **Defensive Programming**
preconditions التحقق من الـ
 - **Assertions**
 - جملة لازم تكون صحيحة عند نقطة معينة في الكود
 - مثال:
assert x >= 0;
 - ممكن نضيف رسالة توضيحية
 - Java مقفولة افتراضياً في assertions مشكلة
-

What to Assert

- Preconditions (مدخلات الميثود)
- Postconditions (نتائج الميثود)
- if / switch الحالات غير المتوقعة في
- تُكتب أثناء كتابة الكود، مش بعده

What NOT to Assert

- الحاجات الخارجية:
 - ملفات
 - Network
 - Input من المستخدم
 - **Exceptions** تُعالج بالـ **bugs** لأن دي مش
 - لا يجب أن تؤثر على منطق البرنامج Assertions
-

Modularity & Encapsulation

- **Modularity**
تقسيم البرنامج لموديولات مستقلة
 - **Encapsulation**
 - استخدام private / public
 - تقليل الوصول للبيانات
 - فوائد:
 - أقل Bugs
 - أسهل Debugging
 - أعلى Reusability
-

Variable Scope

- أسهل Debugging → أصغر scope كل ما الـ
 - Best Practices:
 - for داخل loop تعريف متغير الـ
 - تعريف المتغير عند أول استخدام
 - global variables تجنب الـ
-

Reproduce the Bug

- أول خطوة: حالة صغيرة تعيد إنتاج الخطأ
- ممتاز → regression test لو من

- أصعب في إعادة multithreading أو GUI في Bugs.
-

Understand the Bug (Scientific Method)

1. **Study the data**
 2. **Hypothesize**
 3. **Experiment**
 - print
 - assert
 - debugger
 4. **Repeat**
-

Other Debugging Tips

- في الكود Binary search.
 - ركّز على الكود الجديد.
 - لو متاح component بدّل.
 - تأكد إن الكود محدث.
 - اشرح المشكلة لحد ثاني.
 - 😊 نام كويس
-

Fix the Bug

- افهم هل هو:
 - Coding error
 - Design error
 - اسأل:
 - مشابهة؟ Bugs هل فيه
 - هل الإصلاح هيكسر حاجة ثانية؟
 - bug → **Regression Test.** test أضف
-

Exceptions

What is an Exception?

- حدث يوقف التدفق الطبيعي للبرنامج.
 - يتم:
 - Throw exception
 - Search في call stack
 - Execute handler (catch)
-

Exceptions for Signaling Bugs

- NullPointerException
- ArrayIndexOutOfBoundsException
- ArithmeticException
- NumberFormatException

تدل غالبًا على خطأ برمجي →

Exceptions for Special Results

- 1- أو null أفضل من إرجاع قيم خاصة زي
 - أسهل في القراءة
 - أقل عرضة للنسيان
-

Checked vs Unchecked Exceptions

Checked Exceptions

- compile تُفحص وقت الـ
- لازم:
 - throws
أو
 - try-catch
- مثال: IOException

Unchecked Exceptions

- Runtime Exceptions
 - لا يُجبرك الكومبايلر على التعامل معها
 - Bugs غالبًا
-

Handling Exceptions

- **Method 1:** throws
 - **Method 2:** try-catch (أفضل مع رسائل واضحة)
-

throw Keyword

- يدويًا exception لرمي
- مثال:

throw new ArithmeticException("Error message");

- Custom Exceptions يُستخدم أيضًا مع
-

Custom Exceptions

- خاص بك Exception إنشاء
 - رسالة أوضح
 - أنصف Design
-

Exception Design Considerations

- **Unchecked:** استخدم
 - غير متوقع bug لو
 - أو سهل منعه
 - **Checked:** استخدم
 - لو المستخدم لا يستطيع منعه بسهولة
-

Abuse of Exceptions

- الطبيعي flow للتحكم في الـ exceptions لا تستخدم.
- للحالات الاستثنائية فقط exceptions.